REST ASSURED

Complete API Automation & KT Notes

Java • REST Assured • JUnit/TestNG • Cucumber BDD • Maven • Jenkins

Purpose: A practical, original set of notes for learning REST Assured, giving a team knowledge-transfer session, building a maintainable API automation framework, and preparing for SDET/API testing interviews.

Source alignment: These notes are an original study guide based on the REST Assured topic shown in the supplied Aman Views page/image. They do not reproduce the site's handwritten notes verbatim. The official REST Assured documentation was also checked for current syntax and dependency information.

Prepared for KT / SDET API Automation

Table of Contents

1. API & REST Fundamentals
2. HTTP Methods, Status Codes and Headers
3. REST Assured Overview
4. Maven Project Setup
5. given / when / then Syntax
6. GET Request
7. POST Request
8. PUT, PATCH and DELETE
9. Parameters: Query, Path, Form
10. Headers, Cookies and Content Types
11. Request Body and JSON
12. JSONPath / GPath and Response Validation
13. Response Extraction and Request Chaining
14. POJO Serialization & Deserialization
15. Authentication and Authorization
16. Logging, Debugging and Filters
17. Request/Response Specifications
18. Configuration, Base URI and Reusable Setup
19. JSON Schema Validation
20. Multipart, File Upload and Download
21. XML Testing
22. Negative Testing and Boundary Testing
23. Framework Design for SDET Teams
24. Cucumber BDD Integration
25. Complete Example Project Structure
26. Sample End-to-End API Test Flow
27. Jenkins / CI Execution
28. Best Practices and Common Mistakes
29. KT Delivery Plan
30. Interview Questions & Answers
31. Quick Reference Cheat Sheet

1. API & REST Fundamentals

- Client → sends HTTP request → Server processes it → Server returns HTTP response → Client validates it.
- REST (Representational State Transfer) is an architectural style commonly used with HTTP-based APIs.
- Resources are commonly represented by URLs, for example /users/123 or /orders/5001.
- REST APIs commonly exchange JSON, but XML and other formats are also possible.
- API automation is useful for fast regression, integration validation, contract checks, negative testing and CI/CD.

Typical request

HTTP request

```
POST /api/users HTTP/1.1
Host: example.com
Content-Type: application/json
Authorization: Bearer <token>

{
  "name": "Karthik",
  "role": "tester"
}
```

Typical response

HTTP response

```
HTTP/1.1 201 Created
Content-Type: application/json

{
  "id": 101,
  "name": "Karthik",
  "role": "tester"
}
```

2. HTTP Methods, Status Codes and Headers

Method	Typical purpose	Common success codes
GET	Read resource	200
POST	Create resource / trigger operation	200 / 201 / 202
PUT	Replace/update a resource	200 / 204
PATCH	Partially update a resource	200 / 204
DELETE	Delete resource	200 / 202 / 204
HEAD	Headers only; no response body	200 / 204
OPTIONS	Discover supported operations	200 / 204

Status-code families

Range	Meaning
1xx	Informational
2xx	Successful request
3xx	Redirection
4xx	Client/request-side problem

Range	Meaning
5xx	Server-side problem

- 200 OK — request succeeded.
- 201 Created — a resource was created.
- 202 Accepted — request accepted for asynchronous processing.
- 204 No Content — request succeeded with no response body.
- 400 Bad Request — malformed or invalid request.
- 401 Unauthorized — authentication is missing/invalid.
- 403 Forbidden — identity may be known but access is not allowed.
- 404 Not Found — requested resource/route was not found.
- 409 Conflict — request conflicts with current resource state.
- 422 Unprocessable Content — syntactically valid but semantically invalid input.
- 429 Too Many Requests — rate limit exceeded.
- 500 Internal Server Error — server-side failure.
- 502/503/504 — gateway/service availability or timeout-related failures.

Important headers

Header	Purpose
Content-Type	Format of request body
Accept	Response formats the client accepts
Authorization	Credentials/token
User-Agent	Client identification
Cookie	Session/client state
Cache-Control	Caching behavior
Correlation/Trace ID	End-to-end request tracing in distributed systems

3. REST Assured Overview

Core pattern

```
given()
    .baseUri("https://api.example.com")
    .header("Accept", "application/json")
.when()
    .get("/users/101")
.then()
    .statusCode(200)
    .contentType("application/json");
```

What each section means

Section	Responsibility
given()	Request preconditions: URI, headers, params, auth, cookies, body, etc.
when()	HTTP operation: get(), post(), put(), patch(), delete(), request()
then()	Assertions and extraction: status, headers, body, response time, schema

4. Maven Project Setup

pom.xml - core dependency

```
<dependency>
    <groupId>io.rest-assured</groupId>
    <artifactId>rest-assured</artifactId>
    <version>6.0.1</version>
    <scope>test</scope>
</dependency>

<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter</artifactId>
    <version>${junit.version}</version>
    <scope>test</scope>
</dependency>
```

Recommended project packages

Example layout

```
src/test/java
  ■■■ api
      ■■■ clients
      ■■■ tests
      ■■■ specs
      ■■■ models
      ■■■ utils
      ■■■ constants

src/test/resources
  ■■■ config
  ■■■ testdata
  ■■■ schemas
```

5. given / when / then Syntax

Minimal example

```
given()
    .queryParam("page", 2)
.when()
    .get("/users")
.then()
    .statusCode(200);
```

Common chain methods

Area	Examples
URI	baseUri(), basePath(), port()
Headers	header(), headers()
Parameters	queryParam(), pathParam(), formParam()
Cookies	cookie(), cookies()
Authentication	auth().basic(), auth().oauth2(), auth().preemptive()
Body	body(String), body(Object), body(File)
Validation	statusCode(), header(), contentType(), body(), time()
Debug	log().all(), log().ifValidationFails()
Extraction	extract().path(), response.path(), as(), jsonPath()

6. GET Request

Simple GET

GET + status assertion

```
given()
    .baseUri("https://reqres.in")
.when()
    .get("/api/users/2")
.then()
    .statusCode(200);
```

GET with query parameters

Query parameter

```
given()
    .queryParam("page", 2)
    .queryParam("per_page", 10)
.when()
    .get("/api/users")
.then()
    .statusCode(200)
    .body("page", equalTo(2));
```

Validate multiple response fields

Body assertions

```
then()
    .statusCode(200)
    .body("data.id", equalTo(2))
    .body("data.email", containsString("@"))
    .body("data.first_name", not(emptyOrNullString()));
```

7. POST Request

POST with JSON

```
given()
    .baseUri("https://reqres.in")
    .contentType("application/json")
    .body("{\"name\":\"morpheus\",\"job\":\"leader\"}")
.when()
    .post("/api/users")
.then()
    .statusCode(201)
    .body("name", equalTo("morpheus"))
    .body("job", equalTo("leader"));
```

Using a Map

Map body

```
Map<String, Object> payload = new HashMap<>();
payload.put("name", "morpheus");
payload.put("job", "leader");

given()
    .contentType(ContentType.JSON)
    .body(payload)
.when()
    .post("/api/users")
.then()
    .statusCode(201);
```

8. PUT, PATCH and DELETE

PUT — full replacement/update

PUT

```
given()
    .contentType(ContentType.JSON)
    .body("{\"name\":\"Karthik\",\"role\":\"SDET\"}")
.when()
    .put("/api/users/101")
.then()
    .statusCode(anyOf(is(200), is(204)));
```

PATCH — partial update

PATCH

```
given()
    .contentType(ContentType.JSON)
    .body("{\"role\":\"Senior SDET\"}")
.when()
    .patch("/api/users/101")
.then()
    .statusCode(anyOf(is(200), is(204)));
```

DELETE

DELETE

```
given()
.when()
    .delete("/api/users/101")
.then()
    .statusCode(anyOf(is(200), is(202), is(204)));
```

9. Parameters: Query, Path, Form

Query parameters

Example

```
given()
    .queryParam("status", "ACTIVE")
    .queryParam("limit", 20)
.when()
    .get("/users");
```

Path parameters

Named path parameter

```
given()
    .pathParam("userId", 101)
.when()
    .get("/users/{userId}")
.then()
    .statusCode(200);
```

Form parameters

Form POST

```
given()
    .contentType(ContentType.URLENC)
    .formParam("username", "testuser")
    .formParam("password", "secret")
.when()
    .post("/login");
```

Parameter	Typical location	Use case
Query	URL ?key=value	Filtering, sorting, pagination
Path	URL /users/{id}	Identifying a specific resource
Form	Request body	HTML-style form submissions / application/x-www-form-urlencoded
Body	Request body	JSON/XML/domain payload

10. Headers, Cookies and Content Types

Headers

```
given()
    .header("Accept", "application/json")
    .header("X-Correlation-ID", "abc-123")
    .contentType(ContentType.JSON)
.when()
    .get("/users");
```

Cookies

```
given()
    .cookie("sessionId", "abc123")
.when()
    .get("/profile")
.then()
    .statusCode(200);
```

11. Request Body and JSON

Inline JSON

Readable for a small example

```
String payload = """
{
  "name": "Karthik",
  "role": "SDET"
}
""";

given()
    .contentType(ContentType.JSON)
    .body(payload)
.when()
    .post("/users");
```

Nested JSON

Nested payload

```
{
  "customer": {
    "name": "Karthik",
    "address": {
      "city": "Dublin",
      "country": "Ireland"
    }
  },
  "items": [
    {"sku": "A101", "qty": 2},
    {"sku": "B202", "qty": 1}
  ]
}
```

Best practice

- Use POJOs for stable domain models.
- Use Map for small dynamic payloads.
- Use external JSON files when payloads are large or data-driven.
- Do not hard-code secrets, tokens or passwords in source code.
- Validate both schema and business values where appropriate.

12. JSONPath / GPath and Response Validation

Extracting values

```
Response response =
    given()
```

```
.when()
    .get("/users/2");

int id = response.path("data.id");
String email = response.path("data.email");
```

Useful expressions

Expression	Meaning
data.id	Nested property
data.email	Nested property
data[0].id	First array item's id
data.find { it.id == 2 }.email	Find matching item using GPath
data.size()	Array/list size
data*.id	Collect ids from elements

Hamcrest matchers

Common assertions

```
body("name", equalTo("Karthik"))
body("name", containsString("Kar"))
body("name", notNullValue())
body("roles", hasItem("ADMIN"))
body("roles", hasSize(2))
body("age", greaterThan(18))
body("active", is(true))
```

13. Response Extraction and Request Chaining

Extract an ID

```
String userId =
    given()
        .contentType(ContentType.JSON)
        .body(createPayload)
    .when()
        .post("/users")
    .then()
        .statusCode(201)
        .extract()
        .path("id");
```

Use extracted ID

```
given()
    .pathParam("id", userId)
.when()
    .get("/users/{id}")
.then()
    .statusCode(200)
    .body("id", equalTo(Integer.parseInt(userId)));
```

Response object

Response access

```
Response response =
    given().get("/users/101");

response.statusCode();
response.contentType();
response.header("Content-Type");
response.asString();
response.jsonPath().getString("name");
```

14. POJO Serialization & Deserialization

Request POJO

```
public class User {
    private String name;
    private String job;

    public User() {}

    public User(String name, String job) {
        this.name = name;
        this.job = job;
    }

    // getters and setters
}
```

Serialize + send

```
User user = new User("Karthik", "SDET");

given()
    .contentType(ContentType.JSON)
    .body(user)
.when()
    .post("/users")
.then()
    .statusCode(201);
```

Deserialize response

```
User response =
    given()
.when()
    .get("/users/101")
.then()
    .statusCode(200)
    .extract()
    .as(User.class);
```

15. Authentication and Authorization

Mechanism	Typical REST Assured approach
Basic Auth	auth().basic(username, password)
Preemptive Basic	auth().preemptive().basic(...)
Bearer/OAuth2	auth().oauth2(token) or Authorization header
API Key	header/query parameter according to contract
Session/Cookie	cookie() or SessionFilter
Client certificate	SSL configuration / keystore

Basic authentication

```
given()
    .auth().basic("user", "password")
.when()
    .get("/secure")
.then()
    .statusCode(200);
```

Bearer token

```
given()
    .auth().oauth2(accessToken)
.when()
    .get("/secure")
.then()
```

```
    .statusCode(200);
```

Authorization header

```
given()
    .header("Authorization", "Bearer " + accessToken)
.when()
    .get("/secure");
```

16. Logging, Debugging and Filters

Log request

```
given()
    .log().all()
    .header("Accept", "application/json")
.when()
    .get("/users")
.then()
    .log().all()
    .statusCode(200);
```

Log only on failure

```
then()
    .log().ifValidationFails()
    .statusCode(200);
```

Filters

Conceptual filter

```
public class CorrelationFilter implements Filter {
    @Override
    public Response filter(
        FilterableRequestSpecification req,
        FilterableResponseSpecification res,
        FilterContext ctx) {

        req.header("X-Correlation-ID", UUID.randomUUID().toString());
        return ctx.next(req, res);
    }
}
```

17. Request/Response Specifications

Request specification

```
RequestSpecification requestSpec =
    new RequestSpecBuilder()
        .setBaseUri("https://api.example.com")
        .setBasePath("/api")
        .setContentType(ContentType.JSON)
        .addHeader("Accept", "application/json")
        .build();
```

Response specification

```
ResponseSpecification responseSpec =
    new ResponseSpecBuilder()
        .expectContentType(ContentType.JSON)
        .expectResponseTime(lessThan(3000L))
        .build();
```

Reuse specs

```
given()
    .spec(requestSpec)
.when()
    .get("/users")
.then()
    .spec(responseSpec)
    .statusCode(200);
```

18. Configuration, Base URI and Reusable Setup

Central configuration

```
RestAssured.baseURI = "https://api.example.com";
RestAssured.basePath = "/api";
```

Environment strategy

Environment	Base URL example
DEV	https://dev-api.example.com
QA	https://qa-api.example.com
UAT	https://uat-api.example.com
PROD	https://api.example.com

System property

```
String env = System.getProperty("env", "qa");
// Load environment-specific URL from config.
mvn test -Denv=qa
```

19. JSON Schema Validation

Concept

```
then()
    .body(matchesJsonSchemaInClasspath("schemas/user.json"));
```

Example schema idea

```
{
  "type": "object",
  "required": ["id", "name"],
  "properties": {
    "id": {"type": "integer"},
    "name": {"type": "string"}
  }
}
```

20. Multipart, File Upload and Download

Multipart upload

```
given()
    .multiPart("file", new File("src/test/resources/data.csv"))
    .multiPart("description", "test file")
.when()
    .post("/upload")
.then()
    .statusCode(200);
```

Download response

```
Response response =
    given()
.when()
    .get("/reports/101/download")
.then()
    .statusCode(200)
    .extract()
    .response();

byte[] fileBytes = response.asByteArray();
```

21. XML Testing

XML response validation

```
given()
.when()
    .get("/greeting")
.then()
```

```
    .statusCode(200)
    .body("greeting.firstName", equalTo("John"));
```

XPath example

```
then()
    .body(hasXPath("/greeting/firstName[text()='John']"));
```

22. Negative Testing and Boundary Testing

Scenario	Expected validation
Missing required field	4xx + error contract
Invalid field type	4xx + validation message
Unauthorized	401
Forbidden role	403
Unknown ID	404
Duplicate resource	409 where defined
Too many requests	429 where rate limiting applies
Huge input	Defined validation/limit behavior
Empty string/null	Contract-specific validation
Malformed JSON	400 or documented error

Negative example

```
given()
    .contentType(ContentType.JSON)
    .body("{\"email\":\"not-an-email\"}")
.when()
    .post("/users")
.then()
    .statusCode(400)
    .body("errors.email", not(empty()));
```

23. Framework Design for SDET Teams

Recommended layers

Layer	Responsibility
Tests / Step Definitions	Business scenario and assertions
API Clients	Endpoint-specific request operations
Specifications	Common request/response setup
Models	POJOs for request/response contracts
Utilities	Config, token, data, date, file helpers
Test Data	Externalized JSON/CSV/properties
Reporting	Allure/Extent/custom reporting as required

Example architecture

```
api-automation/
■■■ pom.xml
■■■ src/test/java
■   ■■■ clients
```

```
■   ■   ■■■ UserClient.java
■   ■   ■■■ AuthClient.java
■   ■■■ tests
■   ■   ■■■ UserApiTest.java
■   ■■■ models
■   ■   ■■■ UserRequest.java
■   ■   ■■■ UserResponse.java
■   ■■■ specs
■   ■   ■■■ SpecFactory.java
■   ■■■ utils
■   ■   ■■■ ConfigReader.java
■   ■   ■■■ TokenManager.java
■   ■■■ runners
■■■ src/test/resources
    ■■■ config/
    ■■■ schemas/
    ■■■ testdata/
```

Client example

UserClient

```
public class UserClient {
    public Response createUser(UserRequest request) {
        return given()
            .spec(SpecFactory.requestSpec())
            .body(request)
        .when()
            .post("/users");
    }

    public Response getUser(int id) {
        return given()
            .spec(SpecFactory.requestSpec())
            .pathParam("id", id)
        .when()
            .get("/users/{id}");
    }
}
```

24. Cucumber BDD Integration

Feature file

```
Feature: User API

  Scenario: Create a user
    Given the user API is available
    And I prepare a user with name "Karthik" and role "SDET"
    When I create the user
    Then the response status should be 201
    And the response name should be "Karthik"
```

Step definition concept

```
@When("I create the user")
public void createUser() {
    response = userClient.createUser(request);
}

@Then("the response status should be {int}")
public void verifyStatus(int expected) {
    response.then().statusCode(expected);
}
```

BDD rule

25. Complete Example Project Structure

Project

```
rest-assured-cucumber/
■■■ pom.xml
```

```
■■■ src
■   ■■■ test
■   ■   ■■■ java
■   ■   ■   ■■■ config
■   ■   ■   ■■■ clients
■   ■   ■   ■■■ models
■   ■   ■   ■■■ specs
■   ■   ■   ■■■ steps
■   ■   ■   ■■■ runners
■   ■   ■■■ resources
■   ■       ■■■ features
■   ■       ■■■ schemas
■   ■       ■■■ testdata
■■■ README.md
```

End-to-end flow

- Load environment configuration.
- Create reusable request and response specifications.
- Authenticate and obtain token if required.
- Build request using POJO/test data.
- Call endpoint through API client.
- Validate status code, headers and business response.
- Extract IDs/tokens for downstream calls.
- Clean up created data where the environment requires it.
- Publish report and logs without exposing secrets.

26. Sample End-to-End API Test Flow

Scenario flow

```
// 1. Create
Response create = userClient.createUser(
    new UserRequest("Karthik", "SDET"));

create.then().statusCode(201);

int id = create.path("id");

// 2. Retrieve
Response get = userClient.getUser(id);

get.then()
   .statusCode(200)
   .body("id", equalTo(id))
   .body("name", equalTo("Karthik"));

// 3. Update
Response update = userClient.updateUser(
    id, new UserRequest("Karthik", "Senior SDET"));

update.then().statusCode(anyOf(is(200), is(204)));

// 4. Delete
userClient.deleteUser(id)
    .then()
    .statusCode(anyOf(is(200), is(204)));
```

27. Jenkins / CI Execution

Typical pipeline

- Checkout code from Git.
- Resolve Maven dependencies.

- Select environment through a parameter.
- Run smoke/API regression suite.
- Generate test report.
- Archive logs/screenshots/JSON responses as appropriate.
- Publish results and notify the team.

Maven commands

```
mvn clean test
mvn clean test -Denv=qa
mvn clean test -Dtest=UserApiTest
mvn clean test -Dcucumber.filter.tags="@smoke"
```

CI recommendations

- Use a dedicated service account/test identity.
- Inject secrets securely.
- Use stable test data or controlled setup/teardown.
- Separate smoke tests from full regression.
- Retry only known transient infrastructure failures; do not hide product defects with blind retries.
- Keep API tests independent where possible so failures are diagnosable.

28. Best Practices and Common Mistakes

Best practices

- Keep tests independent and deterministic.
- Centralize base URI and environment configuration.
- Use reusable RequestSpecification and ResponseSpecification.
- Use POJOs for complex request/response contracts.
- Validate status + important headers + business fields.
- Add schema validation for stable contracts.
- Use correlation IDs for distributed-system debugging.
- Mask secrets and personal data in logs.
- Keep test data externalized.
- Make failures actionable with request/response diagnostics.
- Tag smoke, regression and contract tests.
- Run the suite in CI regularly.

Common mistakes

- Hard-coding access tokens in source.
- Putting every implementation detail into Cucumber feature files.
- Using arbitrary sleeps instead of polling for asynchronous APIs.
- Only checking statusCode(200) and ignoring the response body.
- Overusing global mutable RestAssured configuration in parallel execution.
- Writing huge duplicated JSON payloads in every test.
- Logging Authorization headers or cookies.
- Treating all 4xx responses as equivalent.
- Blindly asserting response time with an unrealistic threshold.
- Relying on test order.

29. KT Delivery Plan

Time	Topic	Demo
0–10 min	API/REST basics	Request/response anatomy
10–20 min	REST Assured setup	Maven + first GET
20–35 min	CRUD	GET/POST/PUT/PATCH/DELETE
35–45 min	Assertions	Status/header/body
45–55 min	Params & extraction	Query/path + chaining
55–70 min	Framework	Specs + client layer + POJOs
70–85 min	Cucumber	Feature + step definition
85–95 min	CI	Maven/Jenkins
95–105 min	Interview tips	Common questions + troubleshooting

Suggested live demo

- Start with one GET test so the team sees the DSL.
- Add query/path parameters.
- Add POST with a POJO.
- Extract generated ID.
- Call GET using the ID.
- Update using PATCH.
- Delete the resource.
- Move common setup into RequestSpecification.
- Convert one scenario into Cucumber.

30. Interview Questions & Answers

Q: What is REST Assured?

A: A Java DSL/library for automating and validating REST APIs.

Q: Why use given/when/then?

A: It separates request setup, execution and assertions and makes tests readable.

Q: Difference between query and path parameter?

A: Query parameters filter/modify a request, while path parameters identify a resource in the URL.

Q: How do you validate response JSON?

A: Use then().body() with JSON/GPath expressions and Hamcrest matchers.

Q: How do you extract a token?

A: Use extract().path() or response.jsonPath() and reuse the value in subsequent requests.

Q: How do you pass a bearer token?

A: Use auth().oauth2(token) or an Authorization: Bearer header.

Q: How do you avoid duplicate headers/base URLs?

A: Use RequestSpecification and a central configuration/factory.

Q: How do you validate a JSON schema?

A: Use the JSON Schema Validator module and matchesJsonSchemaInClasspath().

Q: How do you send a file?

A: Use multiPart() for multipart upload; validate the response contract.

Q: PUT vs PATCH?

A: PUT generally represents replacement/idempotent update semantics; PATCH represents partial modification.

Q: 401 vs 403?

A: 401 is authentication-related; 403 means access is not permitted for the authenticated/identified caller.

Q: How do you debug a failed API test?

A: Log request/response safely, inspect status/body/headers, correlation ID, environment and test data.

Q: How do you make tests maintainable?

A: Separate tests, clients, models, specs, configuration and test data.

Q: How do you integrate with Jenkins?

A: Run Maven tests in a pipeline, inject configuration/secrets, publish reports and archive diagnostics.

Q: How do you handle async APIs?

A: Poll a status endpoint/event until a defined condition or timeout rather than using fixed sleeps.

Q: How do you run tests in parallel?

A: Avoid shared mutable state, isolate test data, manage tokens/specs safely and use thread-safe utilities.

Q: What is serialization?

A: Java object → JSON/XML representation.

Q: What is deserialization?

A: JSON/XML response → Java object.

31. Quick Reference Cheat Sheet

Imports

```
import static io.restassured.RestAssured.*;
import static org.hamcrest.Matchers.*;

import io.restassured.http.ContentType;
import io.restassured.response.Response;
import io.restassured.specification.RequestSpecification;
import io.restassured.builder.RequestSpecBuilder;
import io.restassured.builder.ResponseSpecBuilder;
```

CRUD

```
GET     -> when().get("/users/{id}")
POST    -> when().post("/users")
PUT     -> when().put("/users/{id}")
PATCH   -> when().patch("/users/{id}")
DELETE  -> when().delete("/users/{id}")
```

Assertions

```
.statusCode(200)
.contentType(ContentType.JSON)
.header("Content-Type", containsString("json"))
.body("id", equalTo(101))
.body("name", notNullValue())
.body("items", hasSize(greaterThan(0)))
.time(lessThan(3000L))
```

Parameters

```
.queryParam("page", 2)
.pathParam("id", 101)
.formParam("username", "user")
.cookie("session", "abc")
```

Authentication

```
.auth().basic("user", "password")
.auth().oauth2(token)
.header("Authorization", "Bearer " + token)
```

Extraction

```
String token = response.path("token");
int id = response.path("data.id");
String name = response.jsonPath().getString("data.name");
```

Logging

```
.log().all()
.log().headers()
.log().body()
.log().ifValidationFails()
```

Final Revision Checklist

- Explain REST, HTTP and JSON clearly.
- Write GET/POST/PUT/PATCH/DELETE from memory.
- Know queryParam vs pathParam vs formParam.
- Know headers, cookies and content types.
- Write body assertions with Hamcrest.
- Extract IDs/tokens and chain requests.
- Understand serialization/deserialization.
- Know Basic/Bearer/OAuth2/API-key patterns.
- Build reusable request/response specifications.
- Explain POJO + client + test layering.
- Integrate REST Assured with Cucumber.
- Run Maven tests from command line and CI.
- Protect secrets and sensitive data in logs.
- Know negative testing and contract/schema validation.
- Be ready to explain one complete framework from request to Jenkins.

References